

What Went Wrong? Diagnosing Anomalies in Reinforcement Learning Policy Executions

Anonymous submission

Abstract

Reinforcement learning (RL) algorithms output policies specifying which action an agent should take in a given state. However, faults can sometimes arise during policy execution due to internal faults in the agent. As a result, actions may have unexpected effects. In this work, we aim to diagnose such faults and infer their root cause. We call this RL diagnosis problem RLDX. Solving RLDX is useful in practice to facilitate efficient repair of fault agents. We formally define RLDX and design an algorithm for solving it. We then use various techniques to design an improved algorithm. The improved algorithm draws on insights from the model-based diagnosis literatures, extracting conflicts and using them to speedup the diagnosis process. Finally, we evaluate our algorithms on benchmark RL domains and discuss their strengths and limitations. In most cases, our improved algorithm outperforms the baseline, with up to 3 times lower runtime.

Introduction

In this work, we focus on systems consisting of a single autonomous agent. We explore what made an agent diverge from its expected course of action. We focus on a setting where there is no model for the environment. In such cases, a model-free approach is taken to make the agent complete its tasks. In this approach, a *Policy* is used to guide the agent towards executing its tasks while maximizing the rewards obtained by the agent’s interaction with the environment. Such a policy is the product of *Reinforcement Learning* (RL) or other methods. RL models train the policy on data available through simulations or interaction with the environment.

Sometimes, faults may arise during policy execution due to internal failure in the components of the agent. Such faults may cause some of the agent’s actions to have unexpected effects. When this happens, we want to characterize the new behavior that led the agent to execute its task in a faulty manner. This can help repair the components responsible for executing faulty actions faster. In a scenario where repair is not possible, this can improve future planning if the faulty components can be assumed to be faulty, therefore computing suitable policies. To characterize the new behavior of the agent, we propose to use *Diagnosis* techniques.

The field of *Automated Diagnosis* (DX) includes various AI techniques that try to understand the root causes of system failures. The central question in this field is, given

abnormal observations of some system, what causes it to behave differently than expected? Previous work asks this question in a variety of domains such as Software engineering (Abreu, Zoetewij, and Van Gemund 2007), Multi-Agent Systems (Natan and Kalech 2022), and more, and proposes different approaches such as Spectrum-based Fault Localization (Abreu, Zoetewij, and Van Gemund 2007) and Model-Based Diagnosis (Reiter 1987).

In this work, we focus on single agent systems represented as Markov Decision Process (MDP) where the agent uses policy usually generated using RL to determine its actions. The contributions of this work are as follows: (1) We define the problem of diagnosing faulty RL policy executions (RLDX). (2) We propose a novel approach to solving RLDX and present an algorithm called SIF that does it. (3) We propose an approach for improving SIF called SIFU, based on Model-Based Diagnosis (MBD). SIFU uses three techniques for improving SIF: smart selection of which parts of the input to use, smart ordering of which input parts to use first, and early pruning of solutions that are not consistent. (4) We evaluate the algorithms theoretically and empirically on the well-known Gymnasium (Towers et al. 2023) RL domains and discuss their strengths and weaknesses. Results show that SIFU improves SIF in almost every domain we tested with. In some cases, SIFU’s average runtime is less than 33% of SIF’s. An ablation study on the different improvement techniques shows that no single technique consistently outperforms the others, and using their combination gives the most robust algorithm.

Background and Related Work

A *Markov Decision Problem* (MDP) is a sequential decision-making problem in which the cost and transition functions depend only on the current state of the system and the current action (Puterman 1990). Formally, an MDP is a tuple $\langle \mathcal{S}, \mathcal{A}, P_a(S, S'), R_a(S, S') \rangle$ where $\mathcal{S}$ is a set of environment and agent states, $\mathcal{A}$ is the set of actions an agent can perform, $P_a(S, S') = \Pr(S_{t+1} = S' | S_t = S, a_t = a)$ is the probability of reaching state s' given that the current state at time t is S and the agent executes action a . This is also known as the transition function and $R_a(S, S')$ is the immediate reward gained by executing action a from S .

A solution to an MDP is a *policy* that guides the agent to choose actions that maximize its cumulative reward. In

this work, we consider deterministic policies. This means a policy π can be defined as a function $\pi : \mathcal{S} \rightarrow \mathcal{A}$.

Reinforcement Learning

Reinforcement Learning (RL) is a well-studied branch of Machine Learning designed for solving MDPs and other sequential decision-making problems (Sutton and Barto 2018). Some RL algorithms, such as Deep Q-Networks (DQN), learn a function that estimates the value of performing actions in states (Mnih et al. 2015). The policy is computed implicitly by choosing in every state the action that maximizes this value function. Other RL algorithms, such as Asynchronous Advantage Actor-Critic (Mnih et al. 2016) and Proximal Policy Optimization (PPO) (Schulman et al. 2017), learn a policy directly.

The agent applies its actions, based on the learned policy or value function, on an *environment* that defines what the outcomes of those actions are. The environment defines the set of states an agent can be in and the transition function between the states. When an agent in state S executes action a , its next state is determined by the environment.

Model-Based Diagnosis

Automated diagnosis (DX) addresses finding the root cause of a system failure. Model-Based Diagnosis (MBD) is a well-known approach for automated diagnosis. In MBD, the diagnosis problem is defined by a tuple $\langle SD, COMPS, OBS \rangle$ where SD , $COMPS$, and OBS denote the system description, the set of components, and the observations (Reiter 1987). When SD is not consistent with the observation, assuming that all components are healthy, a diagnosis problem arises. In some cases, SD includes information about the faulty operation modes of the system components. This is referred to as having a Strong Fault Model (SFM) (Struss and Dressler 1989). Formally, an SFM defines one or more fault modes for every component. It also specifies how a component is expected to behave in each of its fault modes.

In a given observation, a component $c \in COMPS$ is either healthy, denoted by $h(c)$, or faulty and is in one of its fault modes f_i , denoted by $f_i(c)$. A diagnosis Δ is an assignment of fault modes to components that is consistent with the given observations.

In this work, we address faulty executions of actions in RL environments. Related work focused on diagnosing plan steps that were executed abnormally (Roos and Witteveen 2009; Micalizio and Torasso 2014; Natan and Kalech 2022) while others try to diagnose faulty plan executions that happen due to disagreements between the agents' beliefs (Roos, Ten Teije, and Witteveen 2003; Daigle, Koutsoukos, and Biswas 2006; Kalech and Kaminka 2011). Some work use a single observation (Elimelech et al. 2017), while others use many (Torta and Micalizio 2018). More recent work uses approaches from software diagnosis to diagnose multi-agent systems (Natan, Kalech, and Barták 2023; Natan, Stern, and Kalech 2023). Recent surveys survey other advancements in diagnosis (Kalech and Natan 2022; Rodler 2022).

Previous work explored different approaches for solving MBD problems. An early approach (Reiter 1987) solved the MBD problem by generating a minimal set of conflicts.

Conflicts are sets of components where the assumption they are all healthy is inconsistent with observed faulty behavior, denoted as symptoms. The approach then generates the diagnoses, which are minimal hitting sets of the conflicts. Other notable work includes GDE (De Kleer and Williams 1987), which is an efficient implementation of the classical approach that uses several techniques to focus the diagnosis process, and CDA* (Williams and Ragno 2007), which implements a conflict-directed A* search and generates the conflicts one at a time, any time where the current set of conflicts cannot explain the symptoms.

Problem definition

In this section, we define the RL DX problem. Let Π be an MDP and π a policy created for it, e.g., using some RL algorithm. For simplicity, we assume that the transition function in Π is deterministic. This allows us to define a *simulator* for Π as a function $\chi : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ such that $\chi(S, a)$ is the state the agent is expected to reach when executing a in state S . We extend the definition of χ to address the normal execution of π for i steps starting from S , denoted as $\chi(S, \pi, i)$. This extension is defined as a function $\chi : \mathcal{S} \times \mathcal{A} \times \mathbb{N} \rightarrow \mathcal{T}$ that given a state, an action, and a number of steps returns a trajectory T , where the trajectory is an alternating sequence of actions and states, simulated by the normal execution of the policy π starting from state S .

In this work, we assume executing an action may fail, and there are several *fault modes* that specify different ways in which an action behaves when faulty.

Definition 1 (Fault Mode). *A fault mode is a function $f : \mathcal{A} \rightarrow \mathcal{A}'$ that takes an action $a \in \mathcal{A}$ and returns an action a' from a potentially different set of actions $\mathcal{A}'$. If $\forall a \in \mathcal{A}$ it holds that $f(a) = a$, we say that f is the healthy fault mode, and we denote it as h .*

In this work, we make the common single-fault assumption (Schmid et al. 2020; Perez, Abreu, and d'Amorim 2017), i.e., we assume that only one fault mode is present at any given time. However, a fault mode may affect multiple actions and faulty actions may fail *intermittently*. That is, a faulty action may execute normally in some time steps and faulty in others.

We extend the definition of χ to include simulating a policy together with a fault mode. That means that for a fault mode f , the simulator $\chi(S_0, \pi, i, f)$ outputs a trajectory $T = (S_0, \hat{a}_1, \hat{S}_1, \dots, \hat{a}_i, \hat{S}_i)$ that corresponds to executing the policy from the state S_0 for i steps, where the faulty actions according to f may or may not fail. Because of the intermittency, we extend χ to be a stochastic function, where the trajectory T it returns belongs to the set $\mathcal{T}_{S_0, \pi, i, f}$ of all possible trajectories that can be achieved by executing π from state S_0 for i steps with fault mode f .

Example 1. *Consider the MountainCar environment, a standard RL domain (Towers et al. 2023). Figure 1 provides a visual representation. In this environment, a car is located between two hills. The car can only accelerate to the left, perform no action, and accelerate to the right. We denote these actions as L, N and R. The task of the car is*

to climb the right hill by strategically accelerating while using gravity to build its speed. Let us assume that in a nor-

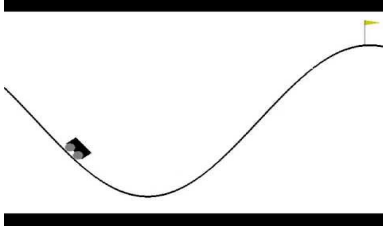


Figure 1: The MountainCar environment.

mal execution, the car would reach its goal and the corresponding trajectory is $(a_1 = R, S_1, a_2 = R, S_2, a_3 = R, S_3, a_4 = N, S_4, a_5 = N, S_5, a_6 = L, S_6, a_7 = L, S_7, a_8 = L, S_8, a_9 = R, S_9, a_{10} = R, S_{10})$. In addition there are 3 candidate fault modes f_1, f_2, f_3 defined as:

- $\mathbf{f}_1(\mathbf{L}) = \mathbf{N}, f_1(\mathbf{N}) = \mathbf{N}, f_1(\mathbf{R}) = \mathbf{R}$
- $f_2(\mathbf{L}) = \mathbf{L}, f_2(\mathbf{N}) = \mathbf{N}, \mathbf{f}_2(\mathbf{R}) = \mathbf{N}$
- $\mathbf{f}_3(\mathbf{L}) = \mathbf{R}, f_3(\mathbf{N}) = \mathbf{N}, f_3(\mathbf{R}) = \mathbf{R}$

Assume the car is faulty and behaves according to f_1 . Because of that, actions a_6 and a_8 failed, and the car did not reach its goal. Also, suppose the car observed the states $\mathcal{O} = (S_0, S_5, S_7, S_{10})$. Because of the failed actions, the states $\hat{S}_7$ and $\hat{S}_{10}$ from the trajectory, returned by simulating $\chi(S_0, \pi, i)$, do not match the observed states S_7 and S_{10} .

In this work we aim to find fault modes that are *consistent* with the observed behavior of the agent when executing a policy π . Let us define this problem formally:

Definition 2 (RLDX problem). An RLDX problem is defined as a tuple $\langle \mathcal{O}, \chi, \pi, \mathcal{F} \rangle$ where π is a policy, χ is a simulator, $\mathcal{O} = (S_0, S_1, \dots, S_n)$ is an observed series of states, and $\mathcal{F}$ is a set of fault modes. An RLDX problem arises when there exists an observed state S_i s.t. $S_i \neq \hat{S}_i$ where $\hat{S}_i$ is the state at step i of the trajectory generated by $\chi(S_0, \pi, i)$.

A solution to an RLDX problem is a *diagnosis*, which in our context is a fault mode f for which there exists a trajectory generated by the execution of $\chi(S_0, \pi, i, f)$ that is consistent with the observations. Formally:

Definition 3 (RLDX Diagnosis). Given an RLDX problem $\langle \mathcal{O}, \chi, \pi, \mathcal{F} \rangle$, a diagnosis is a fault mode f_j such that executing $\chi(S_0, \pi, n, f_j)$ can result in a trajectory $T \in \mathcal{T}_{S_0, \pi, n, f_j}$ where for each observed state $S_i \in \mathcal{O}$ it holds that $S_i = \hat{S}_i$.

Method Description

To address RLDX we present a breadth-first-search approach called SIF, depicted in Algorithm 1. SIF starts by creating a tuple for each fault mode, consisting of the fault mode and the initial state S_0 (lines 1-3). In each step and for each tuple $\langle f_j, \hat{S}_{i-1,j} \rangle$, SIF simulates the next state twice - once by applying the fault mode f_j and once without applying it - in order to support the intermittency of the faults (lines 6-7). If state S_i was not observed, SIF has to take into account the possibility of both generated states. It does so

Algorithm 1: SIF

Input: π - The policy.
Input: χ - The simulator.
Input: $\mathcal{O} = (S_0, \dots, S_n)$ - The observed states.
Input: $\mathcal{F} = \{f_1, f_2, \dots, f_k\}$ - Candidate fault modes.
Result: $\mathcal{D} = \{d_1, d_2, \dots\}$ - The set of diagnoses.

```

1  $\mathcal{G} \leftarrow \emptyset$ 
2 for  $j = 1$  to  $k$  do
3    $\mathcal{G} \leftarrow \mathcal{G} \cup \{\langle f_j, S_0 \rangle\}$ 
4 for  $i = 1$  to  $n$  do
5   for  $\langle f_j, \hat{S}_{i-1,j} \rangle \in \mathcal{G}$  do
6      $\hat{S}_i \leftarrow \chi(\hat{S}_{i-1,j}, \pi(\hat{S}_{i-1,j}))$ 
7      $\hat{S}_{i,j} \leftarrow \chi(\hat{S}_{i-1,j}, f_j(\pi(\hat{S}_{i-1,j})))$ 
8     if  $S_i \notin \mathcal{O}$  then
9        $\mathcal{G} \leftarrow (\mathcal{G} \setminus \{\langle f_j, \hat{S}_{i-1,j} \rangle\}) \cup \{\langle f_j, \hat{S}_i \rangle\}$ 
10       $\mathcal{G} \leftarrow \mathcal{G} \cup \{\langle f_j, \hat{S}_{i,j} \rangle\}$ 
11    else if  $S_i = \hat{S}_i$  then
12       $\mathcal{G} \leftarrow (\mathcal{G} \setminus \{\langle f_j, \hat{S}_{i-1,j} \rangle\}) \cup \{\langle f_j, \hat{S}_i \rangle\}$ 
13    else if  $S_i = \hat{S}_{i,j}$  then
14       $\mathcal{G} \leftarrow (\mathcal{G} \setminus \{\langle f_j, \hat{S}_{i-1,j} \rangle\}) \cup \{\langle f_j, \hat{S}_{i,j} \rangle\}$ 
15    else
16       $\mathcal{G} \leftarrow \mathcal{G} \setminus \{\langle f_j, \hat{S}_{i-1,j} \rangle\}$ 
17  if  $|\mathcal{G}| = 1$  then
18    return  $\{f_j \text{ s.t. } \langle f_j, \hat{S}_{i,j} \rangle \in \mathcal{G}\}$ 
19 return  $\{f_j \text{ s.t. } \langle f_j, \hat{S}_{i,j} \rangle \in \mathcal{G}\}$ 

```

by removing $\langle f_j, \hat{S}_{i-1,j} \rangle$ from $\mathcal{G}$ and adding both states $\hat{S}_i$ and $\hat{S}_{i,j}$, resulting in a total of one new tuple being added to $\mathcal{G}$ (lines 8-10). Otherwise, if S_i is observed, then SIF can compare S_i to $\hat{S}_i$ and $\hat{S}_{i,j}$ and keep only the simulated state that is consistent with S_i . If S_i is equal to $\hat{S}_i$, SIF removes $\langle f_j, \hat{S}_{i-1,j} \rangle$ and adds $\langle f_j, \hat{S}_i \rangle$ only (lines 11-12). If $S_i \neq \hat{S}_i$ but $S_i = \hat{S}_{i,j}$ then SIF removes $\langle f_j, \hat{S}_{i-1,j} \rangle$ and adds $\langle f_j, \hat{S}_{i,j} \rangle$ only (lines 13-14). Otherwise, SIF removes $\langle f_j, \hat{S}_{i-1,j} \rangle$ from $\mathcal{G}$ without adding other tuples (lines 15-16). In every step $1 < i \leq n$, SIF checks the size of $\mathcal{G}$ and halts if the size becomes 1. If SIF had not stopped, there would have been more than one diagnosis.

Example 2. Considering example 1, let us look closely at f_2 , which affects R actions. At the beginning of step $i = 6$ there is only one tuple corresponding to f_2 in $\mathcal{G}$. That is a tuple $\langle f_2, \hat{S}_5 \rangle$ where the state is achieved by not applying f_2 at all, since $\hat{S}_5 = S_5 = \chi(S_0, \pi, 5, h)$. During step $i = 6$ SIF generates for tuple $\langle f_2, \hat{S}_5 \rangle$ two new states - a state $\hat{S}_6$ where action a_6 did not fail and a state $\hat{S}_{6,2}$ where a_6 did fail according to f_2 . Since S_6 is not observed, both tuples $\langle f_2, \hat{S}_6 \rangle, \langle f_2, \hat{S}_{6,2} \rangle$ are added to $\mathcal{G}$ while $\langle f_2, \hat{S}_5 \rangle$ is removed. In the next iteration SIF generates four new states in the same manner, but this time SIF compares them to the observed state S_7 . At this stage, all the tuples of the forms $\langle f_2, \hat{S}_7 \rangle$ and $\langle f_2, \hat{S}_{7,2} \rangle$ will be removed since for every such tuple $\hat{S}_7 \neq S_7$ and also $\chi(S_0, \pi, 7, h) \neq S_7$, meaning that

some other fault mode $f' \neq f_2$ affected an L action.

By assuming intermittent faults, SIF solves a hard problem. This is because SIF has to address the case where an action that should fail according to the fault mode can still be executed normally. This breath-first-search addressing of possible trajectories manifests in expanding the group of fault mode/current state pairs, as shown in lines 9-10 in Algorithm 1. Luckily, such expansion can happen only if the current state is not observed. In fact, once a state S_i is observed for some fault mode f_j , all of the pairs $\langle f_j, \hat{S}_{i-1,j} \rangle$ are removed except for up to one pair $\langle f_j, \hat{S}_{i-1,j}^* \rangle$ for which it holds that the agent executed the previous action in either a normal way, expressed as $S_i = \chi(\pi(\hat{S}_{i-1,j}^*))$ or in a faulty way, expressed as $S_i = \chi(f_j(\pi(\hat{S}_{i-1,j}^*)))$. Formally:

Theorem 1. *Given an observed state S_i , for each $f_j \in \mathcal{F}$ there is at most one tuple of the form $\langle f_j, \hat{S}_i \rangle$ in $\mathcal{G}$.*

Theorem 2. *Given $S_i, S_{i'} \in \mathcal{O}$ s.t. $S_{i+1}, \dots, S_{i'-1} \notin \mathcal{O}$, for each fault mode f_j such that at the end of step i , $\exists \langle f_j, \hat{S}_i \rangle \in \mathcal{G}$, it holds that for every step $i+1 \leq t \leq i'$, there are at most $2^{t-(i+1)}$ new additional tuples $\langle f_j, \hat{S}_t \rangle$ in $\mathcal{G}$.*

Proofs are omitted due to space limitations.

To conclude, when SIF encounters a long sequence of hidden states, the size of $\mathcal{G}$ will grow exponentially in the size of the sequence length, which itself is bounded by n . This is true for each distinct pair $\langle f, \hat{S} \rangle$ in $\mathcal{G}$. In the next section, we show our approach to an upgraded SIF that addresses that.

Improving SIF via Conflict-Directed Diagnosis

RLDX can be cast as a regular MBD problem $\langle OBS, COMPS, SD \rangle$, where OBS is the set of observations $\mathcal{O}$, $COMPS$ includes a single component — the agent, and SD is implicitly defined by the environment simulator and the possible fault modes. The resulting MBD is degenerate since $COMPS$ includes a single component — the whole agent. This prevents an efficient application of MBD algorithms. In addition, our SD includes black-box functions that specify the actions' behavior when healthy and when in different fault modes. By contrast, standard MBD algorithms such as GDE (De Kleer and Williams 1987) and CDA* (Williams and Ragno 2007) require SD to be given in some formal language that allows automated reasoners to infer conflicts. Next, we propose an algorithm called SIFU for solving RLDX that is inspired by conflict-directed MBD algorithms. SIFU exploits the special structure of our problem, efficiently extracting and sorting conflicts to speedup diagnosis generation.

In MBD, a conflict is defined as a set of components where, if all of them are assumed to behave normally, then they are inconsistent with the system description and the observation. In this work, we define a conflict slightly differently: as a range of time steps where, if all actions behaved normally, then the observation is inconsistent with the system description. To define this formally, we first introduce the notion of an *observation gap*:

Definition 4 (Observation Gap). *Given observation $\mathcal{O}$, An observation gap $g = \langle i, j \rangle$ of size m is defined by two differ-*

ent indices $i < j$, holding that $j = i + (m + 1) \wedge S_i, S_j \in \mathcal{O} \wedge \forall i' \in (i, j), S_{i'} \notin \mathcal{O}$.

For brevity, we use the term *gap* instead of observation gap hereonafter. For each gap $\langle \alpha, \beta \rangle$, if we assume that the actions in all of the steps are healthy, then it will follow that in each step $\alpha < \gamma \leq \beta$ the following will be true: $\chi(S_\alpha, \pi, \gamma - \alpha, h) = S_\gamma$. This will be true in particular for step β , for which the state S_β is visible. If, however, for β it holds that $\chi(S_\alpha, \pi, \beta - \alpha, h) \neq S_\beta$ then at least one of the actions was not executed correctly. In that case we say we have a conflict. Formally:

Definition 5 (Conflict). *A gap $g = \langle \alpha, \beta \rangle$ is defined as conflict if $\chi(S_\alpha, \pi, \beta - \alpha, h) \neq S_\beta$.*

In case of a conflict, we are interested to know what actions the agent tried to execute in a normal execution. This allows us to filter out fault modes that do not affect any of the actions of a conflict. Let us define the actions of a conflict:

Definition 6 (Conflict action set). *For a conflict $g = \langle \alpha, \beta \rangle$, the action set A_g is the set of unique actions that were applied during the execution of $\chi(S_\alpha, \pi, \beta - \alpha, h)$.*

We now present SIFU, detailed in Algorithm 2.

Algorithm 2: SIFU

Input: π - The policy.
Input: χ - The simulator.
Input: $\mathcal{O} = (S_0, \dots, S_n)$ - The observed states.
Input: $\mathcal{F} : \{f_1, f_2, \dots, f_k\}$ - Candidate fault modes.
Result: $\mathcal{D} = \{d_1, d_2, \dots\}$ - The set of diagnoses.

- 1 $\mathcal{G} \leftarrow \emptyset$
- 2 **for** $j = 1$ **to** k **do**
- 3 $\mathcal{G} \leftarrow \mathcal{G} \cup \{\langle f_j, S_0 \rangle\}$
- 4 $Gps \leftarrow \{\langle \alpha, \beta \rangle : S_\alpha, S_\beta \in \mathcal{O} \wedge \forall i \in (\alpha, \beta), S_i \notin \mathcal{O}\}$
- 5 $\mathcal{C} \leftarrow \{\langle \alpha, \beta \rangle \in Gps : \chi(S_\alpha, \pi, \beta - \alpha, h) \neq S_\beta\}$
- 6 $\mathcal{C}_{sorted} \leftarrow \text{sort } \mathcal{C} \text{ by increasing gap size}$
- 7 **for** $\langle \alpha, \beta \rangle \in \mathcal{C}_{sorted}$ **do**
- 8 **for** $f_j \in \mathcal{F}$ **do**
- 9 **if** $\forall a \in A_{\langle \alpha, \beta \rangle}, f_j(a) = a$ **then**
- 10 $\mathcal{G} \leftarrow \mathcal{G} \setminus \{\langle f_j, S_0 \rangle\}$
- 11 **for** $\langle \alpha, \beta \rangle \in \mathcal{C}_{sorted}$ **do**
- 12 **for** $i = \alpha + 1$ **to** β **do**
- 13 execute lines 5-16 of SIF
- 14 **if** $|\mathcal{G}| = 1$ **then**
- 15 **return** $\{f_j \text{ s.t. } \langle f_j, \hat{S}_{i,j} \rangle \in \mathcal{G}\}$
- 16 **return** $\{f_j \text{ s.t. } \langle f_j, \hat{S}_{i,j} \rangle \in \mathcal{G}\}$

SIFU starts by creating a tuple for each fault mode, consisting of the fault mode and the initial state S_0 (lines 1-3). Then, it generates the set of gaps Gps (line 4), and the set of conflicts $\mathcal{C}_{sorted}$ sorted by increasing conflict length (lines 5-6). Next SIFU uses $\mathcal{C}_{sorted}$ to filter $\mathcal{G}$ by filtering fault modes that can not explain the conflicts in $\mathcal{C}_{sorted}$ (lines 7-10). From this point, SIFU proceeds to execute the main code of SIF but traverses the conflicts in $\mathcal{C}_{sorted}$ instead of the complete execution length (lines 11-16).

Example 3. *Refer to Example 1. SIFU computes three gaps: $Gps = \{\langle 0, 5 \rangle, \langle 5, 7 \rangle, \langle 7, 10 \rangle\}$ with the respective*

lengths of 4, 1, 2. Next, SIFU computes the conflicts to be $\mathcal{C} = \{\langle 5, 7 \rangle, \langle 7, 10 \rangle\}$. This is because when applying $\chi(S_5, \pi, 7-2, h)$ and $\chi(S_7, \pi, 10-7, h)$ the resulting states $\hat{S}_5$ and $\hat{S}_{10}$ are not equal to the observed states S_7 and S_{10} . Next SIFU sorts $\mathcal{C}$ to get $\mathcal{C}_{sorted} = (\langle 5, 7 \rangle, \langle 7, 10 \rangle)$, and then it filters out the fault mode f_2 . This is because $\mathcal{A}_{\langle 5, 7 \rangle} = \{L\}$, and $f_2(L) = L$, which means that f_2 could not have been the fault mode responsible for $\chi(S_5, \pi, 7-5, h) \neq S_7$. Finally, SIFU needs to compute the diagnosis out of the set $\mathcal{G} = \{\langle f_1, S_0 \rangle, \langle f_3, S_0 \rangle\}$, and it is sufficient to traverse the conflicts in $\mathcal{C}_{sorted}$.

Theoretic analysis

Runtime analysis The components that influence the runtime of both SIF and SIFU are the length between the first and last observed states (n), the number of candidate fault modes (k) and three domain-dependent components: action prediction, action simulation, and state comparison. We refer to those domain dependent components using the constant $R(d)$ where d is the domain name. In the analysis we will focus on the runtime as a function of n and k .

SIF (Algorithm 1) goes over the states and tuples in $\mathcal{G}$, and for each state and tuple it does a finite number of domain-dependent actions. This, combined with theorems 1 and 2, means that the maximum size of $\mathcal{G}$ depends on the hidden states. In the worst case, where only S_0 and S_n are visible, this gives an upper bound of $O(n \cdot 2^{n-1} \cdot k \cdot R(d))$.

SIFU has the same runtime upper bound as SIF in the worst case, where the only observed states are S_0 and S_n . When there are more observed states, the runtime can get better. In scenarios where some of the states are visible, the gaps that are not conflicts are pruned. In addition, some fault modes could be initially crossed out using the conflicts.

Correctness proofs Here we provide some correctness claims. Due to space limitations, we omit some of the proofs.

Theorem 3. *For every fault mode f_j assigned to the agent as diagnosis, the set of trajectories $\mathcal{T}_{S_0, \pi, n, f_j}$ contains a trajectory $T = (S_0, a_1, \hat{S}_1, \dots, a_n, \hat{S}_n)$ such that for every $S_i \in \mathcal{O}$ it holds that $S_i = \hat{S}_i$.*

Theorem 4. *The diagnosis set $\mathcal{D}$ returned by SIFU is the same as the one returned by SIF.*

Proof. To prove Theorem 4 we prove the following. (1) gaps that are not conflicts do not influence the computed diagnosis set. (2) the order in which the conflicts are used does not matter. (3) if a fault mode is filtered then it can not explain the abnormal behaviour of the agent.

(1): recall that a gap $g = \langle \alpha, \beta \rangle$ that is not a conflict holds that $\chi(S_\alpha, \pi, \beta - \alpha, h) = S_\beta$. This means that SIFU will never execute line 16 in Algorithm 1 during traversal of the indices defined by the gap g . Because of that, no fault mode will entirely be removed from $\mathcal{G}$.

(2) given group $\mathcal{G}$ and two conflicts g_1, g_2 , let us denote $\mathcal{G}_{g_2}^{g_1}$ to be the group $\mathcal{G}$ after first executing SIFU for gap g_1 and then executing gap g_2 , and let us similarly denote $\mathcal{G}_{g_1}^{g_2}$. This part states that $\mathcal{G}_{g_2}^{g_1} = \mathcal{G}_{g_1}^{g_2}$. Assume by contradiction that $\exists \langle f, S \rangle \in \mathcal{G}_{g_2}^{g_1}$ s.t. $\langle f, S \rangle \notin \mathcal{G}_{g_1}^{g_2}$. This means that there

exists an action $a \in \mathcal{A}_{g_1} \cup \mathcal{A}_{g_2}$ that was executed incorrectly for $\mathcal{G}_{g_2}^{g_1}$ and for which it holds $f(a) = a$. Since g_1, g_2 are fixed gaps, this means that for $\mathcal{G}_{g_2}^{g_1}$ the same action a was present. In that case, $\langle f, S \rangle$ should have been removed, in contradiction to $\langle f, S \rangle \in \mathcal{G}_{g_2}^{g_1}$.

(3) let there be a fault mode f that was filtered. In other words, $\exists g \in \mathcal{C}$ such that $\forall a \in \mathcal{A}_g, f(a) = a$. Now assume by contradiction that the fault mode can explain the observed behavior. This means that $\forall i$ s.t. $S_i \in \mathcal{O}$ it holds that $\chi(S_0, \pi, i, f) = S_i$. This means that for every conflict $g = \langle \alpha, \beta \rangle \in \mathcal{C}$ it holds that $\chi(S_\alpha, \pi, \beta - \alpha, f) = S_\beta$. However, if that holds, then this means that in every conflict there is an action that is influenced by f . Then, according to lines 7-9 of SIFU, there is no $g \in \mathcal{C}$ such that $\forall a \in \mathcal{A}_g, f(a) = a$, in contradiction to the initial proof assumptions. $\square$

Evaluation

Domains. We experimented with the domains *Acrobot*, *CartPole*, *MountainCar* and *Taxi*, all of which can be found in the Gymnasium website¹. Table 1 shows general statistics about each domain. The “State Vars” row in Table 1 displays the type of variables describing the state — continuous or discrete. The “Env. Type” row displays whether the environment in each domain is dynamic or static, where “dynamic” here means the state may change even if the agent does not do any action, e.g., due to external forces that operate over the agent. The “Avg. Traj.” row displays the average length of the trajectories in this domain.

Domain	Acrobot	CartPole	MountainCar	Taxi
# actions	3	2	3	6
State Vars	Continuous	Continuous	Continuous	Discrete
Env. Type	Dynamic	Dynamic	Dynamic	Static
Avg. Traj.	148	54	183	63

Table 1: General statistics of our benchmark domains.

Experimental setup. The generation of the experiment instances consisted of the following steps: (1) Set the fault mode f_e to be used during policy execution. (2) Set the fault probability p . (3) Execute the policy with the chosen fault mode for up to 200 steps, 10 times. We deemed 200 steps a sufficient trajectory length for the purpose of our experiments, during which at least one fault will occur. (4) Set the percent of visible states v . $v = 0$ means that only S_0 and S_n are visible. (4) Set the number of candidate fault modes k .

Parameters. For our experiments, we considered fault modes that change some of the actions to behave like other actions. We use this type of fault modes for convenience. For each domain except CartPole, we tested 10 different fault modes f_e . For CartPole, we could only test 3 fault modes due to the number of different actions being only two (moving to the left and moving to the right) and the fact that for the experiments we defined fault modes to be *Many-to-one* or *Onto* functions $f : \mathcal{A} \rightarrow \mathcal{A}$. Apart of that, we experimented with the following parameters: $v \in \{30, 40, 60, 80, 100\}$,

¹<https://gymnasium.farama.org/>

	Acrobot					CartPole					MountainCar					Taxi				
	100	80	60	40	30	100	80	60	40	30	100	80	60	40	30	100	80	60	40	30
SIFU-SC	97	107	141	2254	9084	27	32	47	223	3689	93	102	121	384	2346	54	62	77	109	146
SIFU-S	93	101	142	2296	9252	28	33	60	272	4495	96	100	121	454	3257	41	47	62	95	132
SIFU-CF	93	109	165	2570	18746	27	32	47	222	3634	88	100	129	438	2611	54	61	77	109	147
SIFU-C	98	118	185	2690	28717	28	32	48	266	4043	94	110	160	580	3110	54	63	81	116	157
SIFU	93	103	137	2251	9076	27	32	47	223	3630	88	98	118	381	2324	54	61	77	107	144
SIF	93	117	195	2685	28475	28	33	60	309	4584	95	119	185	721	4518	41	49	67	102	141

Table 2: Ablation study table that presents runtimes of SIF and SIFU with the different upgrades.

$p \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$, $k \in \{2, 4, 6, 8, 10\}$. Only in cartpole we experimented with $k \in \{2, 3\}$.

Results. Table 2 presents the runtime in milliseconds with the decrease of visible states percent (second row) for the domains *Acrobot*, *CartPole*, *MountainCar* and *Taxi*. The first four rows (shown in gray) present the results of an ablation study, which we will present later. The last two rows (shown in white) show the runtime for SIFU and SIF. The first thing we observe is the exponential runtime increase with the decrease of observed states. This is a direct result of the intermittent fault setting that SIF and SIFU assume. In each step they have to calculate both $\chi(\hat{S}_{i-1,j}, \pi(\hat{S}_{i-1,j}))$ and $\chi(\hat{S}_{i-1,j}, f_j(\pi(\hat{S}_{i-1,j})))$, which causes $\mathcal{G}$ to grow by a factor of 2^{n-1} . The second thing to be observed is the difference in runtime increase between SIF and SIFU. While both increase exponentially, the runtime of SIFU was observed to increase slower than the runtime of SIF. This shows that the use of conflicts in the diagnosis, together with the order by which they are used and the early pruning of irrelevant fault modes benefits SIFU in reaching slower runtime increase. Third, we can observe that the runtime is different across the different domains. This is because each domain has different actions and environment settings. This difference is visible in particular between the *Taxi* domain and the others, due to the way the environment acts on the agent. In *Acrobot*, *CartPole* and *MountainCar* domains, the state of the agent changes even when the agent does not execute an action due to environmental factors (such as the gravity in *MountainCar*). In the *Taxi* environment, this does not happen. For this reason, many hypothesized state and fault mode duplicate diagnosis pairs are generated, and are not added to $\mathcal{G}$. Last, we can observe that in the *Taxi* domain SIF outperforms SIFU. However, this is due to the relatively long computation time of conflicts, sorting, and fault mode filtering. As we observed, the runtime is different in different domains. For *Taxi* it is so low, that calculating conflicts, sorting them, and filtering fault modes induces relatively high runtime, which makes SIFU slower than SIF. In addition, this runtime difference is very low (3-13 milliseconds).

Figure 2 presents the runtime in milliseconds with the increase in fault probability. We do not observe a consistent trend. We do observe, that SIFU outperforms SIF, mostly when the fault probability is lower. This is expected since lower fault probability means that fewer actions fail. This, in turn, leads to fewer gaps being conflicts, which means that for lower probability values, the number of conflicts SIFU needed is significantly smaller than the number of gaps.

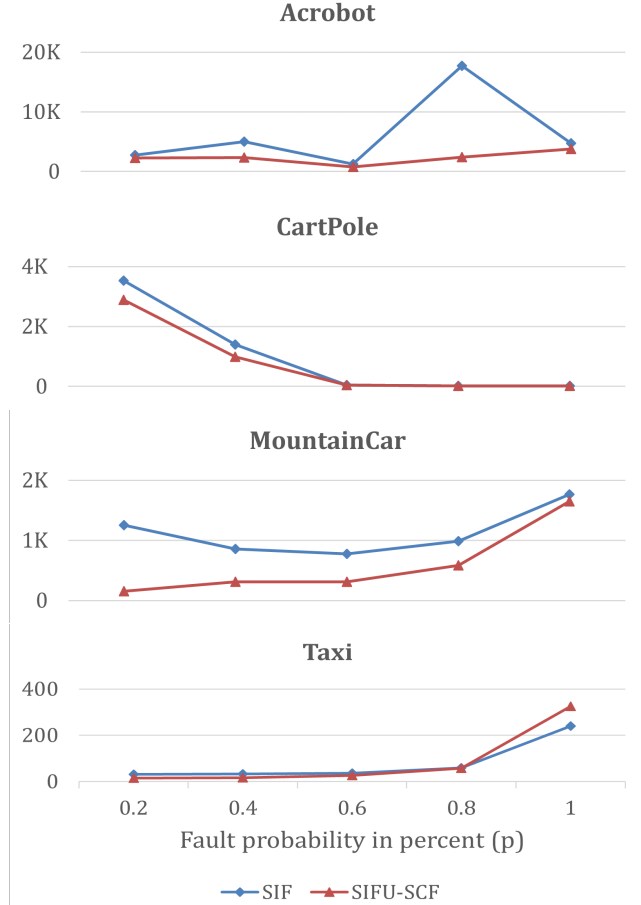


Figure 2: Runtime in milliseconds with increase of p .

Figure 3 shows the increase in runtime with the increase in candidate fault modes. We can observe two things. First, both in SIF and SIFU, the runtime increases with the increase of candidate fault modes. This is an obvious observation since every additional fault mode requires the diagnosis process to do additional computations to determine whether the fault mode is a diagnosis. Second, we can notice that the increase in runtime of SIF is significantly higher than that of SIFU. This is thanks to SIFU's early filtering of fault modes.

Ablation Study SIFU uses 3 enhancements that make it better than SIF — conflict generation, conflict sorting and fault model filtering. In this section we explore how SIFU

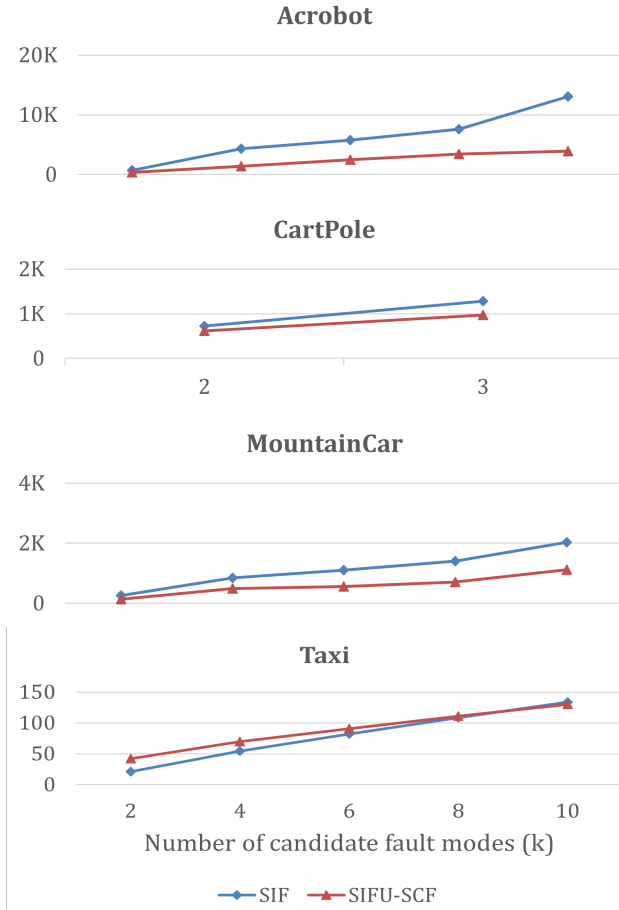


Figure 3: Runtime in milliseconds with increase of k .

changes when we use only some of the enhancements. To that end, let us denote the different enhancements:

- **C** - conflict generation (Line 5 in Algorithm 2).
- **S** - gap sorting (Line 6 in Algorithm 2).
- **F** - fault model filtering (Lines 7-10 in Algorithm 2).

Table 2 presents the runtime in milliseconds for SIF and for SIFU when different components are used. For example, SIFU-SC is a version of SIFU that only calculates conflicts and sorts them. It is not possible to experiment with a SIF version that does only fault model filtering. This is because, in order to filter fault modes, the algorithm has to calculate conflicts first. If a gap $g = \langle \alpha, \beta \rangle$ that is not a conflict would be used, this will make the algorithm filter out all of the candidate fault modes since in gaps, the fault mode is the healthy mode h . In such a case, for every fault mode f_j it will follow that $\chi(S_\alpha, \pi, \beta - \alpha, f_j) \neq S_\beta$.

We can observe that for the domains *Acrobot* and *Taxi*, the sorting of gaps impacts SIFU the most. For *Acrobot*, calculating conflicts and filtering fault modes gives an improvement but is not significant. For *Taxi*, conflict calculation and fault mode filtering actually make SIFU perform worse. We explained the reason in the previous section.

For the *CartPole* and *MountainCar* domains, we observe

that the upgrade responsible for the largest runtime improvement is computing the conflicts. Another interesting thing to note is how the fault mode filter improves SIFU when the gaps are sorted first versus when the gaps are not sorted. When the gaps are sorted, the conflict calculation is the upgrade responsible for the biggest runtime improvement, whereas when the gaps are not sorted, it is the fault mode filtering that is responsible for the largest improvement.

Discussion

Execution fault modes. We generated instances by setting different fault modes for the agent. We defined these fault modes as different *Many-to-one* or *Onto* functions $f : \mathcal{A} \rightarrow \mathcal{A}$. This limits our algorithms twofold. First, some of the candidate fault modes dominate other in terms of function images. This can lead to more than one candidate fault mode being a diagnosis. Second, the manner in which we set the fault modes limits the abnormal behaviour of an agent only to actions that are already defined. This limits our algorithms in addressing problems where a faulty behavior does not manifest as executing a differently defined action.

Randomization of observations. In this work, we use normal distribution to decide which states are observed and which are not. This means that in some occasions, that randomization may lead to long gaps. To mitigate this effect we repeated each experiment 10 times.

Non-Informative policy executions. The policies we executed are all pre-trained policies. As such, we do not have full control on which actions will be chosen during an execution of a plan. Using a different seed in each of the 10 repeats of an instance allowed us to start every time with different starting state S_0 , and by that raising the chance for every action to be executed.

Conclusions and future work

In this work, we define formally the problem of diagnosing faulty executions of policies generated by reinforcement learning algorithms (RLDX), where actions can fail intermittently. We present an algorithm that addresses this problem (SIF). We evaluate its performance both theoretically and empirically, and we propose an upgrade for the algorithm, which is composed of a few parts and draws from MBD concepts (SIFU). We evaluated the algorithms and performed an ablation study to determine the most significant upgrade part for every domain. Our findings are as such: (1) Runtime grows with a decrease in observed states. (2) Depending on which domain, different parts of the algorithm upgrade improve the base algorithm better. (3) There is no significant trend in runtime with the increase in fault probability. However, SIFU outperforms SIF. (4) The runtime increases with the increase in candidate fault modes, and the runtime of SIFU increases slower than that of SIF.

As future work, we plan to research ranking methods for the diagnoses. Additional future research directions involve introducing problems involving multiple agents, lifelong diagnosis utilizing multiple observations and a-priori knowledge, and problems involving hidden fault modes - that are not part of the available modeled fault modes.

References

- Abreu, R.; Zoetewij, P.; and Van Gemund, A. J. 2007. On the accuracy of spectrum-based fault localization. In *Testing: Academic and industrial conference practice and research techniques-MUTATION (TAICPART-MUTATION 2007)*, 89–98. IEEE.
- Daigle, M.; Koutsoukos, X.; and Biswas, G. 2006. Distributed diagnosis of coupled mobile robots. In *Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006.*, 3787–3794. IEEE.
- De Kleer, J.; and Williams, B. C. 1987. Diagnosing multiple faults. *Artificial intelligence*, 32(1): 97–130.
- Elimelech, O.; Stern, R.; Kalech, M.; and Bar-Zeev, Y. 2017. Diagnosing resource usage failures in multi-agent systems. *Expert Systems with Applications*, 77: 44–56.
- Kalech, M.; and Kaminka, G. A. 2011. Coordination diagnostic algorithms for teams of situated agents: Scaling up. *Computational Intelligence*, 27(3): 393–421.
- Kalech, M.; and Natan, A. 2022. Model-based diagnosis of multi-agent systems: A survey. In *AAAI Conference on Artificial Intelligence*, 12334–12341.
- Micalizio, R.; and Torasso, P. 2014. Cooperative monitoring to diagnose multiagent plans. *Journal of Artificial Intelligence Research*, 51: 1–70.
- Mnih, V.; Badia, A. P.; Mirza, M.; Graves, A.; Lillicrap, T.; Harley, T.; Silver, D.; and Kavukcuoglu, K. 2016. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, 1928–1937. PMLR.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533.
- Natan, A.; and Kalech, M. 2022. Privacy-aware distributed diagnosis of multi-agent plans. *Expert Systems with Applications*, 192: 116313.
- Natan, A.; Kalech, M.; and Barták, R. 2023. Diagnosis of intermittent faults in Multi-Agent Systems: An SFL approach. *Artificial Intelligence*, 324: 103994.
- Natan, A.; Stern, R.; and Kalech, M. 2023. Distributed spectrum-based fault localization. In *AAAI Conference on Artificial Intelligence*, 6491–6498.
- Perez, A.; Abreu, R.; and d’Amorim, M. 2017. Prevalence of single-fault fixes and its impact on fault localization. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 12–22.
- Puterman, M. L. 1990. Markov decision processes. *Handbooks in operations research and management science*, 2: 331–434.
- Reiter, R. 1987. A theory of diagnosis from first principles. *Artificial intelligence*, 32(1): 57–95.
- Rodler, P. 2022. How should I compute my candidates? A taxonomy and classification of diagnosis computation algorithms. *arXiv preprint arXiv:2207.12583*.
- Roos, N.; Ten Teije, A.; and Witteveen, C. 2003. A protocol for multi-agent diagnosis with spatially distributed knowledge. In *Proceedings of the second international joint conference on Autonomous agents and multiagent systems*, 655–661.
- Roos, N.; and Witteveen, C. 2009. Models and methods for plan diagnosis. *Autonomous Agents and Multi-Agent Systems*, 19: 30–52.
- Schmid, M.; Gebauer, E.; Hanzl, C.; and Endisch, C. 2020. Active model-based fault diagnosis in reconfigurable battery systems. *IEEE Transactions on Power Electronics*, 36(3): 2584–2597.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Struss, P.; and Dressler, O. 1989. ”Physical Negation” Integrating Fault Models into the General Diagnostic Engine. In *IJCAI*, volume 89, 1318–1323.
- Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement learning: An introduction*. MIT press.
- Torta, G.; and Micalizio, R. 2018. Smt-based diagnosis of multi-agent temporal plans. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, 2097–2099.
- Towers, M.; Terry, J. K.; Kwiatkowski, A.; Balis, J. U.; Cola, G. d.; Deleu, T.; Goulão, M.; Kallinteris, A.; KG, A.; Krimmel, M.; Perez-Vicente, R.; Pierré, A.; Schulhoff, S.; Tai, J. J.; Shen, A. T. J.; and Younis, O. G. 2023. Gymnasium.
- Williams, B. C.; and Ragno, R. J. 2007. Conflict-directed A* and its role in model-based embedded systems. *Discrete Applied Mathematics*, 155(12): 1562–1595.

Reproducibility Checklist

This paper:

Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes)

Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes)

Provides well marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes)

Does this paper make theoretical contributions? (yes)

All assumptions and restrictions are stated clearly and formally. (yes)

All novel claims are stated formally (e.g., in theorem statements). (yes)

Proofs of all novel claims are included. (partial)

Proof sketches or intuitions are given for complex and/or novel results. (yes)

Appropriate citations to theoretical tools used are given. (yes)

All theoretical claims are demonstrated empirically to hold. (yes)

All experimental code used to eliminate or disprove claims is included. (yes)

Does this paper rely on one or more datasets? (no, the Gymnasium (Towers et al. 2023) benchmarks are used)

Does this paper include computational experiments? (yes)
Any code required for pre-processing data is included in the appendix. (NA).

All source code required for conducting and analyzing the experiments is included in a code appendix. (NA)

All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes. (yes)

All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes)

If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results. (NA)

This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks. (no)

This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics. (yes)

This paper states the number of algorithm runs used to compute each reported result. (yes)

Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information. (NA)

The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank). (no)

This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments. (yes)

This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting. (yes)